What is SoapUI?

SoapUI is an open-source API testing tool used to test SOAP and REST web services. It allows us to create and send API requests, validate responses, automate test cases, and perform functional, regression, and security testing. It is mainly used for testing SOAP APIs because it can import WSDL files and automatically generate requests.

SoapUI Key Features

SoapUI provides several features that make API testing efficient:

1. **Supports SOAP and REST APIs** – Used to test both SOAP and REST web services.
2. **WSDL Import** – Automatically imports a WSDL and generates SOAP requests.
3. **Request & Response Validation** – Sends API requests and validates XML or JSON responses.
4. **Assertions** – Verifies status codes, response time, response content, and schema validation.
5. **Test Automation** – Allows creation of reusable test cases and test suites.
6. **Data-Driven Testing** – Runs the same test with multiple sets of input data.
7. **Property Transfer** – Passes values from one request or response to another.
8. **Groovy Scripting** – Supports Groovy scripts for advanced validations and custom logic.
9. **Security Testing** – Tests APIs for common security vulnerabilities.
10. **Mock Services** – Creates mock APIs when the actual service is unavailable.

What SoapUI is used for

- **API Functional Testing** – Verify that APIs return the expected responses.
- **Regression Testing** – Ensure changes don't break existing functionality.
- **Load Testing** (in the Pro version) – Test API performance under heavy traffic.
- **Security Testing** – Check APIs for vulnerabilities.
- **Mock Services** – Create mock API endpoints when the real service is unavailable.

Workflow

"First, we create a SOAP project and import the WSDL. SoapUI automatically generates the services, operations, and sample requests. Then we create a Test Suite and Test Cases. Next, we configure the request by entering the required input data and send the request to the server. After receiving the response, we validate it using assertions. Finally, we execute the Test Suite and analyze the test results."

Simple Steps

1. Create a SOAP Project.
2. Import the WSDL.
3. SoapUI generates services and operations.
4. Create a Test Suite and Test Cases.
5. Configure the request.
6. Send the request.

7. Receive the response.
8. Validate the response using assertions.
9. Execute the Test Suite and analyze the results.

What is WSDL?

WSDL stands for Web Services Description Language. It is an XML document that describes a SOAP web service. It contains all the information required to access the web service, such as the endpoint URL, available operations, request and response formats, and communication protocol. It acts as a contract between the client and the server.

SoapUI uses the WSDL to automatically generate SOAP requests.

What is WADL? (Best Interview Answer)

WADL stands for Web Application Description Language. It is an XML-based document used to describe REST APIs. It provides details such as the API endpoints, supported HTTP methods, request parameters, and response formats, helping clients understand how to interact with the API. However, in modern projects, WADL is rarely used. Most REST APIs use OpenAPI (Swagger) for documentation instead**.**

What is the difference between WSDL and WADL?

WSDL	WADL
Used for **SOAP APIs**	Used for **REST APIs**
XML-based	XML-based
Widely used with SOAP	Rarely used today
Describes SOAP operations	Describes REST resources and HTTP methods

Easy Way to Remember

- **SOAP → WSDL**
- **REST → WADL (older approach)**
- **REST → Swagger/OpenAPI (currently used in most projects)**

What are Assertions?

"Assertions are used to validate the API response against the expected result. They help us verify that the web service is working correctly."

Common Assertions in SoapUI

1. **Contains Assertion** – Checks whether the response contains the expected text.
2. **Not Contains Assertion** – Checks that the response does not contain a specific text.
3. **XPath Assertion** – Validates specific values in an XML response using XPath.

4. **XQuery Assertion –** Performs advanced validation on XML responses using XQuery expressions.
5. **Valid HTTP Status Codes Assertion** – Verifies that the response returns one of the expected HTTP status codes (e.g., **200**, **201**, **204**).
6. **Invalid HTTP Status Codes Assertion** – Verifies that the response does **not** return specific HTTP status codes (e.g., **400**, **404**, **500**).
7. **Script Assertions**: "Script Assertion is used to perform custom validation on XML and JSON responses using Groovy scripting when built-in assertions are not sufficient."
8. **Schema Compliance Assertion** – Verifies that the XML response follows the defined schema (XSD).
9. **SOAP Fault Assertion** – Checks whether the response contains a SOAP Fault.
10. **Response SLA Assertion –** Verifies that the response is received within the expected time.

Sequential and parallel execution

"In SoapUI, test cases can be executed either sequentially or in parallel. In sequential execution, test cases run one after another, and the next test starts only after the previous one finishes. It is suitable for dependent test cases. In parallel execution, multiple test cases run simultaneously, which reduces the execution time. It is suitable for independent test cases."

Documentation in soap ui

"Documentation in SoapUI means documenting the API details, such as the service, operations, requests, responses, test cases, and test results. It helps developers and testers understand the API and makes future maintenance easier."

Xpath

Xpath is a query language for selecting nodes from an xml document.

In **SoapUI XPath Match Assertion**, the commonly used XPath functions include **matches()** and **count()** as well.

XPath Functions in XPath Assertions

1. **contains()** – Checks whether the value contains the specified text.

   ```
   contains(//name,'John')
   ```

2. **matches()** "`matches()` is used to verify whether a value matches the expected pattern or value, such as a name, email ID, phone number, or date format."

   ```
   matches(//name, 'John')
   ```

   ```
   matches(//mobile,'[0-9]{10}')
   ```

   Use: Validate patterns like mobile numbers, email IDs, etc.

3. **count()** – Counts the number of matching XML elements.

   ```
   count(//employee)
   ```

Use: Verify the number of nodes returned.

4. **not()** – Checks that a condition is false.

   ```
   not(//status='Failed')
   ```

5. **starts-with()** – Checks whether a value starts with specific text.

   ```
   starts-with(//name,'Jo')
   ```

6. **ends-with()** *(XPath 2.0 support)*

   ```
   ends-with(//email,'.com')
   ```

What is XQuery? (Interview Answer)

"XQuery stands for XML Query Language. It is used to query, retrieve, and validate data from XML documents. In SoapUI, XQuery Assertion is used to perform complex validations on XML responses."

XQuery FLWOR expressions

FLWOR stands for:

- **F** – **For** → Iterates through XML elements.
- **L** – **Let** → Stores a value in a variable.
- **W** – **Where** → Filters data based on a condition.
- **O** – **Order By** → Sorts the result.
- **R** – **Return** → Returns the final output.

Difference between XPath and XQuery

- **XPath** → Used to locate or select XML elements.
- **XQuery** → Used to query, filter, manipulate, and validate XML data.

Suppose the response is:

```
<employee>
    <name>John</name>
    <salary>50000</salary>
</employee>
```

- **XPath:** Select the name element.

  ```
  //employee/name
  ```

- **XQuery:** Return only employees whose salary is greater than 40000.

  ```
  for $e in //employee
  where $e/salary > 40000
  return $e/name
  ```

What is Script Assertion in SoapUI? (Interview Answer)

Script Assertion is used to write custom validation logic using Groovy scripts. It is useful when the built-in assertions are not sufficient. We can validate both XML and JSON responses using Script Assertions.

For XML Response

{

"id": 101,

"name": "John",

"status": "Success"

}

We use **XmlSlurper** to parse and validate XML.

Example:

```
def xml = new XmlSlurper().parseText(messageExchange.responseContent)
assert xml.name.text() == "John"
```

For JSON Response

We use **JsonSlurper** to parse and validate JSON.

Example:

```
import groovy.json.JsonSlurper

def json = new JsonSlurper().parseText(messageExchange.responseContent)
assert json.name == "John"
```

If you want to **print (log) values** in a SoapUI Script Assertion, use `log.info`.

JSON Example

```
import groovy.json.JsonSlurper

def json = new JsonSlurper().parseText(messageExchange.responseContent)

log.info json.name
log.info json.address.city
```

Output:

```
John
Hyderabad
```

JSON Assertions in SoapUI

1. **JSONPath Match Assertion**
   - **Used to validate the value of a specific JSON element.**
   - Example: Verify `name = "John"`.
2. **JSONPath Existence Match Assertion**
   - **Used to verify that a JSON element exists in the response.**
   - Example: Check whether `address.city` exists.
3. **JSONPath Count Assertion**
   - **Used to verify the number of elements returned by a JSONPath expression.**
   - Example: Verify that there are **5 employees** in the response.
4. **JSONPath RegEx Match Assertion**
   - **Used to verify that a JSON value matches a regular expression or expected pattern.**
   - Example: Validate a **10-digit mobile number** or an **email format**.

What are Properties in SoapUI?

"Properties in SoapUI are used to store and reuse data across test cases and test steps. They help avoid hardcoding values and make the test scripts reusable and easy to maintain."

Types of Properties

1. **Global Properties** – Accessible across all SoapUI projects.
2. **Project Properties** – Accessible throughout a specific project.
3. **TestSuite Properties** – Shared by all test cases within a test suite.
4. **TestCase Properties** – Accessible only within a specific test case.
5. **TestStep Properties** – Accessible only within a specific test step.

In interview say from 2 to 5

How do you access a property?

${#Scope#PropertyName}

where the scope can be Global, Project, TestSuite, TestCase, or TestStep."

Using **Property Expansion**:

```
${#Project#BaseURL}
${#TestSuite#Username}
${#TestCase#CustomerId}
${#TestStep#Token}
${#Global#Environment}
```

How do you access properties in a Groovy Script in SoapUI?",

Project Property

```
def value = context.expand('${#Project#BaseURL}')
log.info value
```

TestSuite Property

```
def value = context.expand('${#TestSuite#Username}')
log.info value
```

TestCase Property

```
def value = context.expand('${#TestCase#CustomerId}')
log.info value
```

Global Property

```
def value = context.expand('${#Global#Environment}')
log.info value
```

TestStep Property

```
def value = context.expand('${#TestStep#Token}')
log.info value
```

Property Transfer

"Property Transfer is a TestStep in SoapUI used to transfer dynamic values from one TestStep to another. It is mainly used when the output of one API becomes the input for another API. This helps handle dependent APIs, avoids hardcoding, and improves test case reusability."

Example

Suppose you have two API requests:

- **Request 1:** Create Employee
  - Response:

```
{
  "employeeId": 101
}
```

- **Request 2:** Get Employee
  - Request:

```
{
  "employeeId": ?
}
```

Instead of manually entering **101**, we use **Property Transfer** to copy `employeeId` from the first response and use it in the second request.

What is SOAP message?

A SOAP Message is an XML document used for communication between a client and a SOAP web service. It consists of four parts: Envelope, Header, Body, and an optional Fault. The Envelope defines the message structure, the Header contains additional information such as authentication, the Body contains the actual request or response, and the Fault provides error details if the request fails."

Here's a simple SOAP Message example that interviewers like.

SOAP Request

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:cus="http://example.com/customer">
   <soapenv:Header>
      <cus:Authentication>
         <cus:Username>admin</cus:Username>
         <cus:Password>admin123</cus:Password>
      </cus:Authentication>
   </soapenv:Header>

   <soapenv:Body>
      <cus:GetCustomer>
         <cus:CustomerId>101</cus:CustomerId>
      </cus:GetCustomer>
   </soapenv:Body>
</soapenv:Envelope>
```

Explanation

- **Envelope** → Root element of the SOAP message.
- **Header** → Contains authentication details (Username & Password).
- **Body** → Contains the actual request (`GetCustomer`) and the `CustomerId`.
- **Fault** → Appears only if an error occurs.

SOAP Fault Example

```
<soap:Fault>
   <faultcode>soap:Client</faultcode>
   <faultstring>Invalid Customer ID</faultstring>
</soap:Fault>
```

1. What is a TestSuite?

"A TestSuite is a collection of related Test Cases used to organize and execute API tests together."

2. What is a TestCase?

"A TestCase is a collection of TestSteps used to validate a specific API functionality."

3. What is a TestStep?

"A TestStep is a single action performed within a TestCase, such as sending a request, performing Property Transfer, or validating a response."

4. What is Property Expansion?

"Property Expansion is a SoapUI feature used to access property values dynamically using the syntax `${#Scope#PropertyName}`. It helps avoid hardcoding and makes test cases reusable."

5. Difference between Property Transfer and Property Expansion?

"Property Transfer is used to transfer dynamic data from one TestStep to another, whereas Property Expansion is used to access stored property values dynamically."

6. What is Mock Service?

"A Mock Service is a simulated API that behaves like the actual service. It is used when the real API is unavailable or still under development, allowing testing to continue without depending on the actual server."

7. What is SOAP Fault?

"SOAP Fault is an XML error message returned by a SOAP web service when a request cannot be processed. It provides details about the error, such as the fault code and fault string."

8. What is SOAP Envelope?

"SOAP Envelope is the root element of a SOAP message. It defines the structure of the message and contains the Header and Body. If an error occurs, it may also contain a SOAP Fault."

9. What are the parts of a SOAP Message?

"A SOAP message consists of four parts: Envelope, Header, Body, and Fault. The Envelope is the root element, the Header contains additional information such as authentication, the Body contains the actual request or response data, and the Fault contains error information."

10. What is Authentication in SoapUI?

"Authentication is the process of verifying the identity of a user or client before allowing access to an API. SoapUI supports authentication methods such as Basic Authentication, Digest Authentication, OAuth 1.0, OAuth 2.0, NTLM, and Kerberos."

11. What is Endpoint?

"An Endpoint is the URL where the API request is sent to access a specific web service."

12. Difference between XPath and JSONPath?

"XPath is used to locate and retrieve data from XML documents, whereas JSONPath is used to locate and retrieve data from JSON documents."

13. What is Data-Driven Testing?

"Data-driven testing is a technique where the same test case is executed multiple times using different sets of input data. In ReadyAPI, it is implemented using DataSource and DataSource Loop."

14. What is Environment Switching?

"Environment Switching allows the same API test cases to run against different environments, such as Development, QA, UAT, and Production, without modifying the test cases."

15. What is TestRunner?

"TestRunner is a command-line utility used to execute SoapUI or ReadyAPI test cases from the command line or through CI/CD tools like Jenkins."

1. What is ReadyAPI?

"ReadyAPI is a commercial API testing tool developed by SmartBear. It is used to test SOAP and REST APIs and provides advanced features such as data-driven testing, security testing, performance testing, reporting, and CI/CD integration."

2. Why do we use ReadyAPI?

"We use ReadyAPI to automate API testing, perform data-driven testing, execute tests in parallel, generate reports, and integrate API tests with CI/CD tools like Jenkins."

3. What are the main features of ReadyAPI?

"ReadyAPI provides functional testing, data-driven testing, security testing, load testing, service virtualization, reporting, environment management, and CI/CD integration."

4. Difference between SoapUI and ReadyAPI?

"SoapUI is an open-source API testing tool, whereas ReadyAPI is its commercial version with advanced features such as data-driven testing, parallel execution, reporting, security testing, load testing, and CI/CD integration."

5. What are the components of ReadyAPI?

"ReadyAPI consists of API Functional Testing, API Performance Testing, API Security Testing, and Service Virtualization."

6. What is DataSource?

"DataSource is used to read test data from external sources such as Excel, CSV, databases, or files and execute the same test with different sets of data."

7. What is DataSource Loop?

"DataSource Loop repeatedly executes the test case until all the records in the DataSource have been processed."

8. What is Environment Switching?

"Environment Switching allows us to execute the same test cases against different environments, such as Development, QA, UAT, and Production, without modifying the test scripts."

9. What is TestRunner?

"TestRunner is a command-line utility used to execute ReadyAPI test cases from the command line or through CI/CD tools like Jenkins."

10. How do you integrate ReadyAPI with Jenkins?

"ReadyAPI can be integrated with Jenkins using TestRunner, allowing API tests to run automatically as part of the CI/CD pipeline."

11.What is DataSink?

"DataSink is a ReadyAPI TestStep used to write or save API response data to external sources such as Excel, CSV, databases, or files. It is mainly used to store execution results or response data."

What is Excel DataSource?

"Excel DataSource is used to read test data from an Excel sheet. Each row in the Excel file is used as input to execute the same API multiple times with different data."

What is JDBC DataSource?

"JDBC DataSource is used to connect to a database and retrieve test data by executing SQL queries. The retrieved data can then be used as input for API testing."

Difference between DataSource and DataSink?

"DataSource is used to read data from external sources, whereas DataSink is used to write or save data to external sources."

Grid DataSource

Grid DataSource is used to store test data inside ReadyAPI in rows and columns. It is useful for small datasets and allows the same test case to run with multiple sets of data without using external files."

Example

"For example, if I want to test a Login API with three different usernames and passwords, I can enter those values directly in a Grid DataSource. ReadyAPI reads each row and executes the same test case for every set of data."